Building a Verifiable Local AI Knowledge System

From a personal case study to a reference architecture: governing ingestion, identity, and provenance in a local-first AI system — not just running models locally.

Alessandro Bellotta

ABSTRACT

Organizations increasingly ask a simple question with a complicated answer: can we get the benefits of AI-assisted knowledge work — search, summarization, tagging, organization — without sending sensitive documents to a third-party service? This paper documents a working answer, built and iterated on over several months: a “second brain” system that watches a personal note-taking vault, ingests documents in common office formats, and makes them semantically searchable, using only AI models that run entirely on a single laptop. Its most durable lesson was not “local AI works” — it does, and the components are freely available today — but that nearly every real failure showed up in ingestion, identity, and provenance, not in the model-facing parts of the system that most RAG tooling optimizes for. A later, deeper round of work turned that finding into something closer to a reference architecture: a small, auditable control plane (an extended manifest recorded per document, a centralized policy module for ingestion decisions, a structured decision log, a model registry, and a deterministic test suite), plus two further adversarial findings — a real symlink-based data-exfiltration path in the ingestion pipeline, and inconsistent-but-real prompt-injection compliance in tag generation — found and fixed by treating the system as something to attack, not just something to use. We map the resulting controls to NIST's AI Risk Management Framework and OWASP's Top 10 for LLM Applications, not as a compliance claim, but as a shared vocabulary for describing what a small, honestly-scoped local AI system can and cannot yet promise. The complete source code is published for reproducibility.

1. Introduction

“Second brain” is the popular name for a simple idea: instead of trusting memory to hold every note, meeting summary, and reference document, keep them in one searchable place and let a system help retrieve and connect them. The natural next step — the one every vendor in this space is now taking — is to point a large language model at that pile of notes so it can search, summarize, and organize on your behalf. That step usually means sending your notes to a cloud API, which is an acceptable trade for a hobbyist's reading list and often not for a consultant's client materials, a lawyer's case files, or an executive's strategy notes.

This project built a working answer to that trade-off: an “AI-assisted second brain” — semantic search, automatic tagging, cross-document linking, daily summaries — delivered entirely by models that never leave the machine they run on. Early use surfaced seven specific failure modes, reported in Section 5, all in ingestion and data hygiene rather than in the language model itself — the first sign that this is where the real engineering difficulty in a system like this actually lives.

A security-oriented review of that early work made a sharper version of the same point: local retrieval-augmented generation is no longer a hard technical problem — the components are commodity — so the differentiator for anyone evaluating a system like this is not “does it run locally” but whether ingestion, identity, versioning, and provenance are actually *governed*, and whether that governance is something a third party can verify rather than something the author merely asserts. That review is the direct cause of the work Section 7 reports: turning a set of ad-hoc, scattered safeguards into a small, coherent control plane, adding the kind of deterministic tests a reviewer (not just the author) can run, and — critically — attacking the system's own ingestion path rather than assuming the existing safeguards were sufficient. One of the two adversarial findings reported there turned up a real, previously unknown vulnerability, which is itself evidence for the review's central claim.

Sections 2–6 describe the system as it was first built and the failure modes found during that phase. Section 7 describes the deeper governance work that followed. The Addendum records the full history of incremental findings in between, in the same “what broke and why” voice throughout.

2. Related Work

This project sits at the intersection of two active threads, and takes a position relative to each. The first is the “LLM wiki” pattern popularized by Andrej Karpathy in 2026: rather than re-deriving answers from scratch on every query, an agent compiles source material once into a maintained markdown knowledge base — several Obsidian-specific forks of that pattern already exist [1]. The system in this paper is a variant of the same idea, but makes a different bet about where the engineering effort belongs. The wiki-pattern projects mostly orchestrate a capable coding agent to write and rewrite pages on request; this system instead runs a small, always-on watcher that reacts to filesystem events and applies deterministic policy — deduplication, format preference, link disambiguation, and now ingestion policy — without invoking a model at all for most of that work. We think this is the more defensible design for something that runs unattended: a background process that only calls a language model for the two tasks language models are actually good at — summarizing and judging relevance — is easier to reason about, and easier to audit, than one that also asks an agent to make and remember structural or security decisions.

The second thread is local retrieval-augmented generation for personal notes, which already has working, shipped tools — Reor [2] and ObsidianRAG [3] being the closest to this project in intent, both pairing Ollama with a local vector store for offline question-answering over a note collection. Where this paper differs is emphasis, now more than ever: those projects, like most RAG tooling generally, optimize the query experience and treat ingestion as a solved preliminary step. This paper's position is closer to the opposite — for a real, multi-year personal or organizational document collection, ingestion is where nearly all of the actual engineering difficulty and, as Section 7 argues, nearly all of the actual *risk* lives, while query-time RAG mechanics are comparatively well understood [6].

Finally, the specific failure described in Section 5.2 — a language model asked to reproduce an exact reference inventing one instead — is not particular to the small model used here. Recent large-scale studies

of LLM-generated citations report similar failure rates in far larger, commercially-deployed models: one legal-domain study measured 13–21% citation hallucination even with retrieval grounding in place [4], and a separate large-scale analysis of structured citation output found only about half of extracted references were valid [5]. Read against this paper's experience, the pattern looks less like an idiosyncrasy of small local models and more like a general property of generative text models used as a source of exact, structured fact.

3. Why Keep It Local?

Three considerations, in order of how often they came up in practice:

Privacy by construction, not by policy. A privacy *policy* is a promise about what a vendor will and won't do with your data. A privacy *architecture* removes the vendor from the loop entirely. When embeddings, search, tagging, and summarization all run on local models, there is no API call carrying document content to audit, no data-processing agreement to negotiate, and no vendor retention window to worry about. The trust boundary collapses to the laptop itself — which, as Section 7 argues, is necessary but not sufficient: a local trust boundary still needs its own internal governance.

Cost that doesn't scale with usage. A system that re-indexes a growing archive and re-evaluates it continuously makes a meaningful number of AI calls per day, indefinitely. Metered cloud APIs turn that into a running bill that grows with exactly the behavior you want to encourage. Local models, once downloaded, are effectively free to run.

No external dependency for uptime or rate limits. The system works on a train, in a client's basement server room with no signal, or during a provider outage.

The honest trade-off is capability and speed: local models, especially on a laptop without a dedicated GPU, are smaller and slower than the frontier cloud models. Section 5 treats that trade-off as a central design constraint, not a footnote.

4. What the System Does

At a high level, the system has four moving parts:

1. **Ingestion.** Documents (PDF reports, Word documents, PowerPoint decks, plain text, and now audio recordings) are dropped into a folder, or pointed to in an external location such as a shared drive. The system extracts the readable content into a structured note, and — as of this paper's revision — evaluates an ingestion *policy* before doing so (Section 7).
2. **Indexing.** Each note is broken into passages, and each passage is converted into a numerical representation (an “embedding”) by a small local model, stored in a local vector database.
3. **Organization.** A second, slightly larger local model reads each new note alongside the notes it's most semantically similar to, and proposes short tags and links between related material.
4. **Retrieval.** A search box and a conversational chat mode (available from the command line, from inside the note-taking application, and now from any MCP-compatible AI client — Section 7) turn a plain-language question into an answer grounded in the vault's own content.

None of this requires a network connection once the models are downloaded.

5. A Case Study in What Broke

Each of the following was a working feature that was removed, replaced, or redesigned after real use exposed a problem.

5.1 A local vision model confidently hallucinated on business diagrams

An early version of the system extracted images embedded in slide decks and used a small local vision model to caption them. The model produced empty output in non-English prompts, and — more seriously — confidently invented plausible-sounding but wrong descriptions of real business diagrams. **A wrong answer stated with total confidence is more dangerous than no answer at all**, because it gets stored and treated as fact by everything downstream. The feature was removed rather than shipped as "mostly working."

5.2 Never let a generative model be the source of truth for a fact that must be exactly correct

The tag-and-link feature originally asked the model to write the actual link text pointing to a related note, as free-form output; it occasionally invented paths that didn't exist. **Ask the model to choose from a numbered list, and have ordinary code construct the actual fact from the chosen option.** The model judges; the code transcribes. This division of labor is a cheap, general-purpose safeguard for any system where LLM output will be used as a reference or a piece of structured data.

5.3 Two real things can share the same name

Two genuinely different files in the document collection happened to share an identical filename in different folders — a common occurrence when teams reuse a template's default filename. **"Assume unique filenames" is not a safe assumption** in a real document repository; the system now detects the collision and falls back to a fully qualified reference only when actually needed.

5.4 A safety-motivated design choice quietly doubled the system's complexity

An early safeguard wrote AI suggestions into a companion file mirroring the original vault's folder structure, to avoid ever silently modifying hand-written content — correct for hand-written notes, but applied uniformly even to notes the system itself had generated, where there was nothing to protect. **A safeguard's scope should match the actual risk it addresses**, not be applied blanket-wide for simplicity.

5.5 Duplicate documents need an explicit, automatic policy

Real document collections accumulate near-duplicates (`report_v1.pptx`, `report_vFINAL.pptx`, a PDF export of the same deck). Two deterministic rules — keep the highest version, prefer PDF over an Office-format duplicate — resolved nearly every real instance, with no model involved. **Most of the "garbage in, garbage out" problem in practice is version sprawl and format duplication**, and responds well to simple, deterministic policy.

5.6 A claim about data locality needs to be verified technically, not assumed from configuration

A routine check revealed the vault's folder had been silently redirected into a cloud-storage sync location by the operating system's own “back up your folders” feature, months earlier. AI *processing* had been entirely local, as designed; *storage* had not been, and those are two different guarantees. **Verify where the files actually live on disk; don't infer it from where you told the software to put them.**

5.7 Real-time reactivity is not always the right default

For content the system doesn't own — a shared drive, a colleague's export — continuous real-time watching is both unnecessary and mildly risky. The system now treats externally-owned content as a daily, read-only, never-auto-deleted archive by default. **Reactivity should be proportional to ownership and volatility.**

6. What This Suggests for Organizations Evaluating Local AI

- **It is genuinely feasible today, on ordinary hardware, for text-centric tasks.**
- **Local AI capability is currently uneven across tasks, not uniformly “behind” cloud AI.** Text was reliable enough to build on; image understanding, at the same size class, was not.
- **The riskiest failures were quiet, not loud.** None of Section 5's problems caused a crash. Section 7 extends this into an operating principle: assume the next quiet failure is still out there, and go looking for it deliberately.
- **Data-locality claims are a technical fact to verify, not a policy statement to trust.**

7. Toward a Verifiable Control Plane

Sections 5 and 6 describe a system that behaved correctly by default and needed correcting when it didn't — a case study read *after the fact*. This section describes the opposite posture, adopted in a later phase of the same project after a security-focused review of the work so far: assume the existing safeguards are incomplete, and go find out how, before someone else does.

7.1 An extended, per-document manifest

Every generated note already carried a small amount of provenance in its frontmatter — which source file produced it, when. That manifest was extended to record, for every document: a content hash, the fully resolved filesystem path it was actually read from, its detected MIME type, the version of the extraction logic that processed it, the embedding model and chunking strategy used, an ingestion timestamp, the ingestion policy decision (below), and — where applicable — an OCR or speech-to-text confidence score. None of this requires a database: it is written as ordinary frontmatter, alongside the note it describes, so a note's own provenance travels with it rather than living in a system that can drift out of sync with the content it describes.

7.2 A single ingestion policy module, not scattered checks

Path-escape checks, size limits, and format allowlisting previously lived in different files, applied inconsistently (a size limit existed for externally-indexed folders but not for the primary ingestion path, for instance). A single policy module now evaluates every candidate document twice — once on its path and size, before any content is read, and once on its extracted text — and returns one of three decisions: allow, quarantine, or deny. Quarantine is the interesting case: a document whose text matches a secret- or PII-like pattern (an API token, an email address, a card-like digit sequence) is still written as a note and recorded in the index by path and content hash, but is never sent to the embedding model — it becomes findable by exact reference, not by semantic search, which is a meaningfully smaller exposure. This same module also checks, once per watcher startup, whether the vault's own storage location resolves through a known cloud-sync path — turning Section 5.6's one-time manual discovery into a standing, automatic check.

7.3 Attacking the ingestion path, not just the model

Section 5.2's safeguard — the model only ever picks an index into a code-controlled list — was extended with a deliberate adversarial test: a source document containing text aimed at the model rather than a human, instructing it to emit a tag matching a wikilink pattern or containing a raw newline. Tested in English, the model complied, producing a tag that would have rendered as a real, clickable link in the note. The same test in Italian did not convince it — model resistance to injection is language-dependent and probabilistic, which is itself the reason it cannot be the only defense. A content-level sanitizer now rejects any tag containing wikilink syntax, newlines, or the note's own internal marker sequence, regardless of whether the model was fooled.

A second, more consequential test targeted the ingestion path itself rather than the model: a symlink placed inside the primary document-drop folder, pointing at a file entirely outside the vault. Before this test, the system read through the symlink without complaint, extracted the external file's content, wrote it into a note, embedded it, and indexed it — at that point genuinely indistinguishable, in search or chat, from something the user had deliberately provided. The cause was mundane — the standard library call used to read a file follows symbolic links by default, and nothing checked where a path actually resolved to before reading it — which is exactly why it had gone unnoticed: nothing about it looked unusual until someone went looking specifically for it. The fix rejects any candidate document whose resolved path falls outside the intended source folder. This generalizes past one bug: a system whose core promise is “only documents you deliberately provide are processed” is making a claim about *provenance*, and that claim is a testable, falsifiable one — not something to take on faith once the code has been written.

7.4 A deterministic test suite, and an honest account of its limits

A small test suite now covers the policy module, the extended manifest, and the tag sanitizer with ordinary, deterministic unit tests — no model call involved — runnable in a few seconds and wired into continuous integration on every change. This deserves an honest caveat rather than an overstated one: hosted CI runners have no local model to call, so this suite cannot and does not test retrieval quality or a model's susceptibility to injection — those remain local, on-demand checks (a retrieval-quality harness against a small synthetic fixture vault, and the adversarial tests from Section 7.3), run by a person, not a pipeline. A canary-document check complements this: it re-runs a fixed document through tag generation and compares the result against a saved baseline using tag-set similarity rather than exact match, since a local model's sampling is not fully deterministic — the point is to flag *significant* drift after a model or prompt change for a human to look at, not to demand bit-for-bit reproducibility a probabilistic system cannot honestly offer.

7.5 A minimal model registry, and an interface for future connectors

Ollama already computes and exposes a content digest for every model it manages; a small registry now records, for each model this system actually depends on, that digest alongside its role (embedding, tag generation, digest writing) and any operator notes — answering “which exact model produced this note's tags” without inventing a new hashing scheme. Separately, a minimal `Connector` interface now describes the contract a future read-only external source (a cloud-drive API, a wiki export) would need to satisfy to plug into this system's ingestion model, with the existing filesystem-based external-folder indexer as its one concrete, working implementation. This is deliberately an interface, not a shipped set of connectors: each additional source (SharePoint online, Google Drive, Notion, a Slack or Teams export) needs its own authentication, pagination, and rate-limit handling — real, separate engineering, not a variation on this one — and is left as explicit future work rather than claimed as done.

7.6 An explicit instruction/data boundary, and a client-facing interface

Every call to the local language model previously concatenated instructions and untrusted note content into a single prompt string. Calls now use Ollama's chat-completion interface with explicit message roles — instructions in a `system` message, retrieved or extracted content always in a `user` message — which most instruct-tuned models weight differently. Retested after the change, this did not eliminate the injection result from Section 7.3 (the same English-language test still succeeded), which is the honest finding: this is a mitigation that raises the cost of a specific class of injection, not a guarantee that removes the risk. The conversational chat mode also gained real multi-turn structure (prior turns as actual messages, not a flattened text block) as a side effect of the same change. Finally, the system is now reachable through the Model Context Protocol, exposing search, note retrieval, a grounded question-answering tool, the daily digest, and a health check to any MCP-compatible client — a standard interface in place of a bespoke one, at no cost to the safeguards described above, since the MCP layer calls the same retrieval and generation code directly rather than reimplementing it.

7.7 A shared vocabulary, not a compliance claim

Table 1 maps the controls above — together with several already present before this review — to NIST's AI Risk Management Framework [7] (organized around four functions: *Govern*, *Map*, *Measure*, *Manage*) and to categories from OWASP's Top 10 for LLM Applications [8]. The mapping is offered as a shared vocabulary for describing what this system does, in terms a security reviewer already uses, not as a claim of conformance — a single-maintainer prototype tested by one person is not an audited system, and saying otherwise would repeat exactly the overclaiming this paper argues against elsewhere. Several of the most consequential findings in this project — the cloud-sync surprise in Section 5.6, the symlink escape in Section 7.3 — are not LLM-specific risks at all; they are ordinary application- and data-security concerns that happen to matter enormously in an AI pipeline, and the table is honest about which rows are “classic” LLM risks versus general engineering discipline wearing an AI hat.

Control	What it addresses	NIST AI RMF	OWASP LLM Top 10
Ingestion path resolution (§7.3)	Symlink/path escape reading unintended files	Manage	Not LLM-specific — data-ingestion integrity
Cloud-sync detection (§7.2, §5.6)	Undisclosed data-locality/storage risk	Govern, Map	Not LLM-specific — infrastructure concern
Secret/PII quarantine (§7.2)	Sensitive content reaching the embedding model	Manage, Map	Sensitive Info Disclosure; Vector/Embedding Weaknesses
Numbered-candidate links (§5.2)	Model fabricating exact structured references	Manage	Improper Output Handling
Tag content sanitizer (§7.3)	Model output injecting structure into notes	Manage	Improper Output Handling
System/user role separation (§7.6)	Instructions embedded in untrusted content	Manage	Prompt Injection
Adversarial tag/chat testing (§7.3)	Assumed vs. actual behavior under attack	Measure	Prompt Injection
Retrieval + canary checks (§7.4)	Silent quality regression after a change	Measure	Not LLM-specific — model quality assurance
Model registry with digest (§7.5)	“Which exact model produced this output”	Govern, Map	Adjacent to Supply Chain (model provenance)
Structured decision/event log (§7.2, §7.6)	After-the-fact review, incident response	Measure, Manage	Not LLM-specific — general audit logging

Table 1. Controls mapped to NIST AI RMF and OWASP LLM Top 10 (descriptive, not a conformance claim).

7.8 A note on regulatory context

This section is descriptive, not legal advice. Under the EU AI Act, governance obligations for general-purpose AI models began applying from 2 August 2025, with broader enforcement and transparency obligations for applicable rules following from 2 August 2026 [9]. A system like this one is not itself a general-purpose AI model provider, but an organization deploying local AI tooling built on one sits inside a regulatory environment that increasingly expects exactly the kind of artifact this section describes — a record of what was ingested, under what policy, using which model, with what was found when someone tried to break it — rather than an assertion that everything is fine. That is the frame this paper suggests for a system like this going forward: not only a privacy tool, but a verifiable local AI knowledge system.

8. Limitations and Future Work

This is a single-user prototype, developed and tested by one person, not a hardened multi-user product. The reliability numbers throughout are anecdotal or drawn from a single small benchmark, not a large-scale study. Section 7's controls were themselves built and tested by the same person who built the system they govern — real independent review, an external audit, or a red team with no stake in the outcome would be a meaningfully stronger form of verification than anything reported here, and is explicitly not what this paper claims to offer. The Connector interface (§7.5) has exactly one concrete implementation; whether it actually generalizes to an API-based source with authentication and pagination

is untested. The local vision-model gap from Section 5.1 remains a fast-moving area. The intent, as before, is not to claim these specific numbers or choices are permanent, but to document failure and hardening *shapes* — silent hallucination, model-as-transcriber, unverified locality, unverified provenance, injection resistance that varies by language — likely to recur in any similar system.

9. Conclusion

The first phase of this project established that local AI's hard problems live in ingestion and data hygiene, not in the model. The deeper work in Section 7 adds a corollary: governing those problems well enough that a skeptical outsider could check the work, rather than take the author's word for it, is itself most of what “production-ready” should mean for a system like this — and it is achievable, at small scale, without standing up infrastructure disproportionate to a project one person can actually maintain. The symlink vulnerability in Section 7.3 is the clearest evidence for this paper's argument: it existed, unnoticed, through every prior round of “what broke,” until the system was deliberately treated as something to attack rather than something to use. That shift — from case study to something closer to a reference architecture — is the actual contribution here, more than any individual control in Table 1.

Reproducibility

The complete source code for the system described in this paper — the document ingestion pipeline, the policy and manifest layer, the MCP server, the deduplication logic, the note-taking application plugin, and the daily digest generator — is published under an open-source license at:

github.com/alebellotta/second-brain-control-plane

The repository intentionally omits any actual document content, personal file paths, or organization-specific configuration; it is meant to be read and adapted, not run unmodified against someone else's files. An earlier, simpler snapshot of this codebase, before the control-plane work in Section 7, remains available at github.com/alebellotta/local-second-brain for anyone who wants to see the system before that phase of the work.

Addendum: A History of Follow-Up Findings

This paper went through several rounds of follow-up work after its initial release, each recorded here rather than silently folded into the sections above, in the order they actually happened.

Round 1 — Closing gaps flagged as open earlier in this paper

First, Section 8 (then Section 7) noted that reliability claims throughout were anecdotal rather than benchmarked. A small retrieval-evaluation harness now measures this directly: a fixed set of (question, expected note) pairs is run through the same retrieval path a user experiences, reporting Precision@1/@3/@5 and mean reciprocal rank. On this system's real note collection, the first run scored Precision@1 of 33% and Precision@3/@5 of 67% — and, more usefully than the numbers themselves, it surfaced a concrete failure this paper could previously only describe in the abstract: two pairs of genuinely different documents about the same underlying event sometimes rank each other's content above the correct match.

Second, PDF pages with no extractable text layer now fall back to local OCR instead of being silently skipped.

Third, the image-captioning experiment from Section 5.1 was deliberately repeated against a second, different local vision model (`llava-phi3`, 3.8B parameters) on the same real slide images. The result was not an improvement — an independent replication of the same failure shape, with a different but equally confident hallucination in English and degenerate, non-empty garbage output in Italian (arguably worse, since it looks like output rather than an obvious non-answer). Two independent models failing in related ways is stronger evidence of a genuine capability boundary than one model failing once.

Round 2 — Adding capability, not just fixing gaps

Fourth, a benchmark ran the identical tag/link-suggestion task against the system's primary model at three quantization levels (Q4_K_M, Q8_0, fp16) on the same consumer laptop CPU. The speed gap was large and as expected — Q4_K_M roughly three times faster than fp16 — but quality did not improve monotonically with precision on this specific task. “Use the highest-precision model your hardware can run” is not a safe default to assume; it needs checking per task.

Fifth, free-text parsing of the model's tag/link output was replaced with Ollama's native structured-output support, constraining generation to a JSON schema. This does not change the numbered-candidate safeguard from Section 5.2; it removes a smaller, separate failure mode where a model would almost-but-not-quite follow a requested free-text format.

Sixth, the system gained a conversational RAG chat mode alongside one-shot semantic search, synthesizing direct answers from retrieved context and supporting follow-up questions, with the same discipline as Section 5.2: sources shown to the user are the passages the retrieval code actually selected, never a citation the model produced itself.

Round 3 — A security dimension, and new application areas

Seventh, the numbered-candidate safeguard was deliberately stress-tested from an adversarial angle: an embedded instruction convinced the English-language model to emit a tag matching a wikilink pattern, which would have rendered as a real link — the same finding now folded into Section 7.3. The identical test in Italian failed to convince the model, establishing that injection resistance is language-dependent and cannot be the only defense; a content-level sanitizer now closes the gap regardless.

Eighth, the ingestion pipeline gained audio transcription as a new source type, using a local speech-to-text model, validated end-to-end with synthetic speech but not against real multi-speaker, noisy recordings.

Ninth, a small local model was fine-tuned (LoRA, on-device via Apple's MLX framework) on a real, small note collection. The mechanics worked cleanly — consistent validation-loss improvement, a small fraction of the model's parameters touched, a compact adapter file, roughly fifteen minutes on a consumer laptop — but the clearest qualitative change was the model reproducing the ingestion pipeline's own markdown conventions rather than a personal authorial voice, a direct consequence of what a corpus dominated by auto-extracted business documents actually contains, and a training/inference format mismatch that limited how well the effect generalized to ordinary chat use.

Tenth, a security pass found the most consequential issue in this entire history: a symlink placed inside the primary ingestion folder let the system silently read, index, and expose the content of a file entirely outside

the vault. This is folded into Section 7.3 above as the clearest single piece of evidence for this paper's current thesis.

Round 4 — From ad-hoc safeguards to a control plane

The fourth round of follow-up work, prompted by the security-oriented review described in Section 1, is what Section 7 documents in full: the extended per-document manifest, the unified policy module (with its secret/PII quarantine and automated cloud-sync detection), a deterministic test suite wired into continuous integration, a canary-document regression check, a minimal model registry keyed on Ollama's own digests, a `Connector` interface with one working implementation, the migration to explicit system/user message roles, real multi-turn chat history, an MCP server, and privacy-preserving structured event logging (operation names, durations, model identifiers, and counts — never note content — in place of free-text log lines, adopting OpenTelemetry's vocabulary without adopting its infrastructure). Section 7 is the fuller account; it is referenced here only to keep this addendum's numbering complete and in order.

References

[1] Karpathy, A. (2026). *LLM Knowledge Bases* [gist]. gist.github.com/karpathy/442a6bf555914893e9891c11519de94f

[2] *Reor* — private, local AI personal knowledge management app [software]. github.com/reorproject/reor

[3] *ObsidianRAG* — privacy-first RAG for Obsidian notes using local AI [software]. github.com/Vasallo94/ObsidianRAG

[4] Citation Grounding: Detecting and Reducing LLM Citation Hallucinations via Legal Citation Graphs (2026). *arXiv:2606.00898*. arxiv.org/pdf/2606.00898

[5] GhostCite: A Large-Scale Analysis of Citation Validity in the Age of Large Language Models (2026). *arXiv:2602.06718*. arxiv.org/pdf/2602.06718

[6] Gao, Y. et al. Retrieval-Augmented Generation for Large Language Models: A Survey (2023/2026). *arXiv:2312.10997*. arxiv.org/abs/2312.10997

[7] National Institute of Standards and Technology. *AI Risk Management Framework (AI RMF 1.0)*. nist.gov/itl/ai-risk-management-framework

[8] OWASP Foundation. *OWASP Top 10 for Large Language Model Applications*. genai.owasp.org/llm-top-10

[9] European Commission. *EU AI Act — Implementation Timeline*. ai-act-service-desk.ec.europa.eu/en/ai-act/eu-ai-act-implementation-timeline